SECULOGIX

Embedded IoT & Electronics Hardware

InStock Inventory Management System

Full-Depth Professional Implementation Plan

v1.0 | June 2026 | CONFIDENTIAL

Prepared For	SecuLogix Engineering Team
Document Type	Technical Implementation Plan
System	InStock — Parts & Component Inventory
Stack	Next.js 14 + PostgreSQL + Prisma ORM
Status	Draft — Awaiting Client Inputs

Table of Contents

1. Project Overview

SecuLogix InStock is a full-featured web-based inventory management system purpose-built for embedded IoT and electronics hardware companies. It replaces ad-hoc spreadsheets with a structured, searchable, location-aware database of every electrical, electronic, IoT, and general part in your facility.

1.1 What This System Does

- Categorised stock entry: Electrical, Electronic (Module / Component / Device), IoT, General
- HSN code capture on every item for GST compliance
- Bin-level storage location tracking (Zone > Rack > Shelf > Bin)
- Interactive dashboard: real-time stock counts, low-stock alerts, category breakdown
- Instant search: find any part by name, part number, package, or location
- Full stock movement audit trail (IN / OUT / TRANSFER)
- Role-based access: Admin and Operator logins

1.2 Target Users

Role	Responsibilities	Access Level
Admin	Full CRUD on all stock, users, locations, HSN codes	All modules
Operator	Add/view stock, search, record movements	No user management, no delete
Viewer	Read-only: view stock levels and locations	Dashboard + Search only

2. Information Required from SecuLogix

> *Before development begins, the following inputs must be confirmed. Fill in and return this section to the development team.*

2.1 PostgreSQL Database

Parameter	Description	Your Value
DB_HOST	Server IP or hostname	_____
DB_PORT	Default: 5432	_____
DB_NAME	Database name (e.g. seculogix_db)	_____
DB_USER	PostgreSQL username	_____
DB_PASSWORD	PostgreSQL password	_____
DB_SSL	SSL required? (yes/no)	_____

2.2 Hosting / Deployment Environment

Question	Your Answer
Deployment target	[ ] Local Ubuntu Server [ ] VPS (DigitalOcean/Linode) [ ] AWS/GCP/Azure [ ] Vercel + Supabase
Domain / IP	_____
Node.js version on server	_____ (required: 18+)
OS on server	_____
Nginx / reverse proxy?	[ ] Yes [ ] No

2.3 Authentication & Users

Question	Your Answer
Total number of users	_____
Login method	[ ] Username + Password [ ] Email + Password [ ] Google SSO
Who is the first Admin?	Name: _____ Email: _____
Session timeout	_____ hours (recommended: 8 hrs)

2.4 Warehouse / Lab Location Structure

> *The system supports a 4-level location hierarchy. Confirm how many zones, racks, shelves, and bins exist in your facility.*

Level	Example	Your Configuration
Zone	Component Lab, Electrical Store	_____
Rack	R-01, R-02 …	_____
Shelf	S-01, S-02 …	_____
Bin	B-01, B-02 …	_____

2.5 Additional Feature Decisions

Feature	Include? (Yes / No / Later)
Barcode / QR Code label printing	_____
Export to Excel / CSV	_____
Low-stock email / SMS alerts	_____
Stock request / purchase workflow	_____
Vendor / supplier master	_____
Multi-site / multi-warehouse support	_____
Dark mode UI	_____

3. Technology Stack

3.1 Core Technologies

Layer	Technology	Why Chosen
Frontend + API	Next.js 14 (App Router)	Full-stack React, SSR, API routes, file-based routing — ideal for internal tools
Database	PostgreSQL 15	Open-source, ACID compliant, excellent JSON support for flexible fields
ORM	Prisma ORM	Type-safe queries, auto-migration, excellent DX with PostgreSQL
UI Library	shadcn/ui + Tailwind CSS	Professional component library, fully customisable, no lock-in
Authentication	NextAuth.js v5	Secure session management, credential + OAuth providers
State Management	Zustand + React Query (TanStack)	Lightweight client state + optimistic server-state caching
Charts / Graphs	Recharts	React-native, responsive charts for dashboard widgets
Form Handling	React Hook Form + Zod	Type-safe form validation aligned with Prisma schema types
Icons	Lucide React	Clean, consistent icon set for electronics/hardware context
Search	PostgreSQL Full-Text Search	No extra service needed; fast tsvector search on parts table

3.2 Supporting Tools

- ESLint + Prettier — code quality and formatting
- Husky + lint-staged — pre-commit hooks
- dotenv / .env.local — environment variable management
- ts-node — TypeScript execution for seed scripts
- PostgreSQL pg_dump — database backups
- PM2 — process manager for production deployment (if self-hosted)

4. System Architecture

4.1 High-Level Architecture

> *The InStock system uses a monolithic Next.js application with a PostgreSQL database backend. There are no microservices — keeping it simple, maintainable, and deployable to a single server.*

The request flow for a typical stock lookup:

#	Layer	Responsibility	Technology
1	Browser	Renders React UI, manages local form state, shows real-time search results	Next.js / React
2	API Routes	Handles REST requests, validates JWT session, applies Zod schema	/app/api/*
3	Service Layer	Business logic: compute low-stock, aggregate stats, format responses	TypeScript modules
4	Prisma ORM	Translates TypeScript queries to SQL, manages schema migrations	Prisma Client
5	PostgreSQL	Stores all data; runs FTS search via tsvector indexes	PostgreSQL 15

4.2 Authentication Flow

- **Login:** User submits credentials → NextAuth verifies against bcrypt hash in users table → JWT issued with role claim.
- **Session:** JWT stored in HttpOnly cookie, 8-hour expiry. Middleware protects all /dashboard/* routes.
- **Authorisation:** Role checked on every API route. ADMIN can DELETE; OPERATOR can INSERT/UPDATE; VIEWER is read-only.

5. Database Schema (PostgreSQL)

> *All tables use UUID primary keys for security and portability. Timestamps are stored in UTC with timezone.*

5.1 Table: users

Column	Type	NN	Notes
id	UUID	Y	PK, default gen_random_uuid()
name	VARCHAR(120)	Y	Full display name
email	VARCHAR(255)	Y	UNIQUE — used as login identifier
password_hash	TEXT	Y	bcrypt hash, never stored in plaintext
role	ENUM	Y	ADMIN | OPERATOR | VIEWER
is_active	BOOLEAN	Y	Soft-disable without deleting account
created_at	TIMESTAMPTZ	Y	Default now()
updated_at	TIMESTAMPTZ	Y	Auto-updated via trigger

5.2 Table: hsn_codes

Column	Type	NN	Notes
id	UUID	Y	PK
code	VARCHAR(20)	Y	UNIQUE — e.g. 8542, 8536, 8471
description	TEXT	Y	GST description of the commodity
gst_rate	NUMERIC(5,2)	N	GST % (e.g. 18.00, 12.00)
created_at	TIMESTAMPTZ	Y	Default now()

5.3 Table: stock_categories

Column	Type	NN	Notes
id	UUID	Y	PK
name	VARCHAR(80)	Y	UNIQUE: electrical | electronic | iot | general
label	VARCHAR(120)	Y	Display label: Electrical, Electronic, IoT, General
icon	VARCHAR(80)	N	Lucide icon name shown in UI
sort_order	INTEGER	Y	UI ordering

5.4 Table: stock_subcategories

Only relevant for Electronic category: Module, Component, Device. Other categories have no subcategory.

Column	Type	NN	Notes
id	UUID	Y	PK
category_id	UUID	Y	FK → stock_categories.id
name	VARCHAR(80)	Y	module | component | device
label	VARCHAR(120)	Y	Module, Component, Device

5.5 Table: locations

Column	Type	NN	Notes
id	UUID	Y	PK
zone	VARCHAR(80)	Y	e.g. Component Lab, Electrical Store
rack	VARCHAR(40)	N	e.g. R-01
shelf	VARCHAR(40)	N	e.g. S-03
bin	VARCHAR(40)	N	e.g. B-07
label	TEXT GENERATED	Y	Computed: 'Component Lab > R-01 > S-03 > B-07'
description	TEXT	N	Optional free-text notes about this location
is_active	BOOLEAN	Y	Default true — can disable empty locations

5.6 Table: parts (Core Inventory Table)

> *This is the central table. One row = one distinct part/SKU. All category-specific fields are nullable columns; only the relevant ones are populated based on category.*

Column	Type	NN	Notes
id	UUID	Y	PK
name	VARCHAR(200)	Y	Human-readable part name
part_number	VARCHAR(100)	N	Manufacturer or internal part number
category_id	UUID	Y	FK → stock_categories
subcategory_id	UUID	N	FK → stock_subcategories (Electronic only)
hsn_code_id	UUID	Y	FK → hsn_codes
package	VARCHAR(60)	N	Component only: 0402, 0603, SOT-23, SOP-8, DIP-16 …
comment	TEXT	N	Free-text notes, datasheet link, supplier info
image_url	TEXT	N	Optional: stored in /public/parts/ or S3

Column	Type	NN	Notes
datasheet_url	TEXT	N	URL to manufacturer datasheet PDF
created_by	UUID	Y	FK → users.id
created_at	TIMESTAMPTZ	Y	Default now()
updated_at	TIMESTAMPTZ	Y	Auto-updated
search_vector	TSVECTOR	N	FTS column: auto-populated by trigger on name + part_number

5.7 Table: stock_entries

One row per part-per-location. A single part can be stored across multiple locations.

Column	Type	NN	Notes
id	UUID	Y	PK
part_id	UUID	Y	FK → parts.id
location_id	UUID	Y	FK → locations.id
quantity	INTEGER	Y	Current stock count (always >= 0)
unit	VARCHAR(30)	Y	pcs, rolls, metres, sets, kg …
min_quantity	INTEGER	N	Low-stock alert threshold
updated_at	TIMESTAMPTZ	Y	Last quantity change time

UNIQUE constraint: (part_id, location_id) — prevents duplicate entries for same part in same bin.

5.8 Table: stock_movements (Audit Log)

Column	Type	NN	Notes
id	UUID	Y	PK
part_id	UUID	Y	FK → parts
from_location_id	UUID	N	NULL for IN movements
to_location_id	UUID	N	NULL for OUT movements
movement_type	ENUM	Y	IN | OUT | TRANSFER | ADJUSTMENT
quantity	INTEGER	Y	Units moved
reference	VARCHAR(120)	N	PO number, project code, or reason
notes	TEXT	N	Free-text reason / remark
performed_by	UUID	Y	FK → users.id
performed_at	TIMESTAMPTZ	Y	Timestamp of movement

5.9 PostgreSQL Indexes

Index Name	On Table / Column	Purpose
idx_parts_search	parts.search_vector	Full-text search GIN index
idx_parts_pn	parts.part_number	Fast part number lookup
idx_parts_category	parts.category_id	Filter by category
idx_stock_location	stock_entries.location_id	Location-based queries
idx_movements_part	stock_movements.part_id	Audit history per part
idx_movements_date	stock_movements.performed_at	Date-range filtering for reports

6. Feature Specifications

6.1 Dashboard — Portfolio View

The dashboard is the landing page after login. It gives an at-a-glance view of the entire inventory health.

6.1.1 Widget Cards Row

Card	Data Source	Visual
Total Parts (SKUs)	COUNT(*) FROM parts	Number + trend arrow
Total Stock Units	SUM(quantity) FROM stock_entries	Number badge
Low Stock Alerts	quantity < min_quantity	Red badge count
Categories	4 fixed categories	Donut chart breakdown
Recent Activity	Last 10 stock_movements	Timeline list

6.1.2 Category Portfolio Cards

One expandable card per category showing part count, total units, and a mini bar chart of subcategory breakdown. Clicking opens filtered parts list.

6.2 Stock Category System & Form Fields

> *Category determines which fields appear on the Add Stock form. All categories share common fields; category-specific fields appear dynamically.*

Common Fields (All Categories)

Field	Input Type	Validation
Part Name	Text	Required, min 2 chars, max 200 chars
Part Number	Text	Optional — alphanumeric + hyphens
HSN Code	Searchable Dropdown	Required — select from hsn_codes table
Comment / Notes	Textarea	Optional, max 2000 chars
Location	Cascading Dropdown	Required — Zone > Rack > Shelf > Bin
Quantity	Number	Required, integer >= 0
Unit	Dropdown	Required: pcs, rolls, metres, sets, kg, g, ml, l
Min Quantity	Number	Optional — triggers low-stock alert

Category: Electrical

Category-specific additional fields for electrical parts (wires, fuses, switches, connectors, MCBs, relays, etc.):

Field	Input Type	Notes
Voltage Rating	Text	e.g. 230V AC, 12V DC — optional
Current Rating	Text	e.g. 5A, 10A — optional
Datasheet URL	URL	Link to PDF datasheet — optional

Category: Electronic > Module

Modules are complete boards: ESP32, Raspberry Pi Pico, relay boards, motor drivers, display modules, etc.

Field	Input Type	Notes
Manufacturer	Text	e.g. Espressif, STMicroelectronics — optional
Datasheet URL	URL	Link to product page or datasheet — optional

Category: Electronic > Component

Discrete components: resistors, capacitors, inductors, transistors, ICs, diodes, etc. Package is critical for component identification.

Field	Input Type	Notes
Package	Searchable Dropdown	REQUIRED for components — see package list below
Value / Specification	Text	e.g. 10kΩ, 100nF, 5V 1A — optional
Tolerance	Text	e.g. 1%, 5%, 10% — optional
Datasheet URL	URL	Manufacturer datasheet — optional

Standard Component Packages supported in dropdown:

- SMD Passives: 0201, 0402, 0603, 0805, 1206, 1210, 1812, 2512
- SOT: SOT-23, SOT-23-5, SOT-23-6, SOT-89, SOT-223
- IC SMD: SOP-8, SOIC-8, SOIC-16, QFP-32, QFP-44, TQFP-48, TQFP-64, LQFP-100
- IC THT: DIP-8, DIP-14, DIP-16, DIP-28, DIP-40
- Through-Hole: TO-92, TO-220, TO-247, DO-35, DO-41
- BGA / QFN: QFN-16, QFN-32, QFN-48, BGA-256 (and custom entry)

Category: Electronic > Device

Completed sub-assemblies, PCBs, test equipment, dev boards:

Field	Input Type	Notes
Serial Number	Text	Optional — for serialised assets
Manufacturer	Text	Optional

Field	Input Type	Notes
Firmware Version	Text	Optional — for tracked devices

Category: IoT

IoT modules, sensors, actuators, gateways, antennas, SIM modules, GPS trackers, etc.

Field	Input Type	Notes
Communication Protocol	Multi-select	WiFi, BLE, Zigbee, Z-Wave, LoRa, MQTT, NB-IoT, LTE-M, RS485, CAN
Operating Frequency	Text	e.g. 433 MHz, 2.4 GHz, 868 MHz — optional
Manufacturer	Text	e.g. Quectel, SIMCom, u-blox — optional
Datasheet URL	URL	Optional

Category: General

Consumables, tools, packaging, cables, PCB materials, fasteners, and other miscellaneous items:

Field	Input Type	Notes
Sub-label	Text	Free user-defined subcategory, e.g. 'Cable', 'Tool', 'PCB'
Supplier	Text	Optional — vendor name

6.3 Add Stock — Step-by-Step Wizard

The Add Stock flow is a 4-step wizard to reduce errors and guide the user:

Step	Name	What Happens
1	Select Category	Large icon tiles for: Electrical / Electronic / IoT / General. If Electronic: sub-step selects Module / Component / Device.
2	Part Details	Dynamic form renders the correct fields for chosen category. Part name, part number, specifications, comment, HSN code, datasheet URL.
3	Location & Qty	Cascading location dropdowns (Zone > Rack > Shelf > Bin). Quantity, unit, min-quantity threshold.
4	Review & Save	Summary of all entered data. User confirms → saved to parts + stock_entries + stock_movements (movement_type: IN).

> *If the same part already exists (detected by part_number match), the wizard offers 'Update Quantity' instead of creating a duplicate.*

6.4 Search & Location Finder

The search page is accessible from the top nav bar at all times. It provides:

- **Instant Search Bar:** Searches across part name, part number, comment, package, HSN code simultaneously using PostgreSQL full-text search + ILIKE fallback.
- **Filter Panel:** Category, subcategory, package type, location zone, HSN code, quantity range, low-stock only toggle.
- **Results Table:** Shows part name, part number, category, package, total quantity, and all storage locations with bin-level detail.
- **Location Card:** Clicking any result opens a side panel showing complete location breakdown: Zone > Rack > Shelf > Bin with quantity at each.
- **Quick Actions:** From results: Edit Part, Add Quantity, Move Stock, View History buttons.

6.5 Stock Movement — IN / OUT / TRANSFER

Type	Use Case	Behaviour
IN	New purchase received, found stock	Increases quantity in stock_entries, logs to stock_movements
OUT	Used in project, damaged, scrapped	Decreases quantity; blocks if would go below 0; logs movement
TRANSFER	Moving part between bins/racks	Decreases source location, increases destination, single transaction
ADJUSTMENT	Physical count correction	Sets absolute quantity with reason note; admin only

7. Next.js Project Structure

The project follows Next.js 14 App Router conventions with a clean domain-driven structure:

Path	Purpose
/app/(auth)/login	Login page — NextAuth credential form
/app/(dashboard)/page.tsx	Main dashboard with stat widgets
/app/(dashboard)/parts/	Parts list, detail, and add wizard pages
/app/(dashboard)/parts/add/	Multi-step Add Stock wizard
/app/(dashboard)/search/	Search & location finder page
/app/(dashboard)/movements/	Stock movement log and forms
/app/(dashboard)/locations/	Location management (admin)
/app/(dashboard)/hsn/	HSN code master management (admin)
/app/(dashboard)/users/	User management (admin only)
/app/api/auth/[...nextauth]/	NextAuth handler
/app/api/parts/	CRUD API for parts — GET, POST, PUT, DELETE
/app/api/stock/	Stock entries and movement APIs
/app/api/search/	Search endpoint — full-text + filter
/app/api/dashboard/	Dashboard aggregates API
/lib/prisma.ts	Prisma client singleton
/lib/auth.ts	NextAuth config + session callbacks
/components/ui/	shadcn/ui base components
/components/parts/	PartsTable, PartCard, AddPartWizard
/components/dashboard/	StatsCard, CategoryChart, MovementTimeline
/components/search/	SearchBar, FilterPanel, LocationCard
/prisma/schema.prisma	Prisma schema — all models defined here
/prisma/seed.ts	Seed: categories, subcategories, HSN codes, default admin
/prisma/migrations/	Auto-generated SQL migration files

8. API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/auth/signin	Public	NextAuth credential login
GET	/api/dashboard	All roles	Stat cards, category breakdown, recent movements
GET	/api/parts	All roles	List parts with pagination, filter, sort
POST	/api/parts	OPERATOR+	Create new part
GET	/api/parts/[id]	All roles	Single part with stock entries
PUT	/api/parts/[id]	OPERATOR+	Update part fields
DELETE	/api/parts/[id]	ADMIN only	Soft-delete part (set is_deleted flag)
GET	/api/search	All roles	Full-text search with filters
GET	/api/stock	All roles	Stock entries, optionally filtered by part/location
POST	/api/stock/movement	OPERATOR+	Record IN / OUT / TRANSFER / ADJUSTMENT
GET	/api/locations	All roles	List all locations (tree structure)
POST	/api/locations	ADMIN only	Create location
GET	/api/hsn	All roles	List HSN codes for dropdown
POST	/api/hsn	ADMIN only	Add new HSN code
GET	/api/users	ADMIN only	List all users
POST	/api/users	ADMIN only	Create user with hashed password
PUT	/api/users/[id]	ADMIN only	Update role, name, active status

9. UI / UX Design System

> *The InStock UI is built around the SecuLogix brand: dark, precision, industrial — inspired by embedded electronics and IoT hardware. It feels like mission-critical software, not a generic SaaS tool.*

9.1 Colour Palette

Token	Hex Value	Usage
--color-bg	#0D0D0D	App background — near-black, matches SecuLogix logo
--color-surface	#1A1A1A	Cards, panels, sidebars
--color-border	#2D2D2D	Card borders, dividers
--color-primary	#F5A623	SecuLogix orange — buttons, badges, active states
--color-primary-dark	#C8780A	Button hover states
--color-text-primary	#F0F0F0	Headlines, labels
--color-text-secondary	#9A9A9A	Body text, captions
--color-success	#22C55E	Stock OK, saved confirmations
--color-warning	#EAB308	Low-stock alerts
--color-danger	#EF4444	Out-of-stock, delete actions
--color-info	#3B82F6	Info badges, links

9.2 Typography

Element	Font / Size	Notes
Page Title (H1)	Inter 600 / 28px	White
Section Heading (H2)	Inter 500 / 20px	SecuLogix orange #F5A623
Card Title	Inter 500 / 16px	Light gray #F0F0F0
Body / Labels	Inter 400 / 14px	#9A9A9A secondary gray
Stat Numbers	JetBrains Mono / 32px	Monospace for technical data; orange
Part Numbers	JetBrains Mono / 13px	Inline code style for part numbers

9.3 Navigation Structure

The application uses a persistent left sidebar with icon + label navigation:

Icon	Label	Links To
LayoutDashboard	Dashboard	/dashboard — stat overview
Plus	Add Stock	/dashboard/parts/add — wizard
Package2	All Parts	/dashboard/parts — searchable table
Search	Find Stock	/dashboard/search — full search + location
ArrowLeftRight	Movements	/dashboard/movements — movement log
MapPin	Locations	/dashboard/locations — zone/rack/bin mgmt
Tag	HSN Codes	/dashboard/hsn — HSN master table
Users	Users (Admin)	/dashboard/users — visible to ADMIN only

10. Implementation Phases & Timeline

> *Estimated timeline assumes one experienced full-stack developer. Timeline can be compressed with 2 developers working in parallel.*

Phase 1 — Foundation (Week 1–2)

#	Task	Days	Owner
1.1	Initialise Next.js 14 project with TypeScript, Tailwind, ESLint	0.5	Dev
1.2	Set up PostgreSQL instance and create database	0.5	Dev + SecuLogix
1.3	Configure Prisma — write schema.prisma, run first migration	1	Dev
1.4	Install and configure shadcn/ui with SecuLogix dark theme	1	Dev
1.5	Configure NextAuth.js — credential provider, session, middleware	1	Dev
1.6	Build left sidebar navigation shell + layout wrapper	1	Dev
1.7	Write seed script: categories, subcategories, HSN codes, admin user	1	Dev

Phase 2 — Core Inventory Features (Week 3–5)

#	Task	Days	Owner
2.1	Parts CRUD API — GET / POST / PUT / DELETE with Zod validation	2	Dev
2.2	Add Stock Wizard — Step 1: Category selector with icons	1	Dev
2.3	Add Stock Wizard — Step 2: Dynamic field forms per category	2	Dev
2.4	Add Stock Wizard — Step 3: Location cascading dropdowns + quantity	1.5	Dev
2.5	Add Stock Wizard — Step 4: Review & confirm with save transaction	1	Dev
2.6	HSN code searchable dropdown with type-ahead	0.5	Dev
2.7	Parts list page — table with sort, filter, pagination	1.5	Dev
2.8	Part detail page — full info + current stock locations	1	Dev

Phase 3 — Search & Stock Movements (Week 6–7)

#	Task	Days	Owner
3.1	PostgreSQL FTS — search_vector trigger + GIN index	1	Dev
3.2	Search API endpoint — full-text + filter query builder	1.5	Dev
3.3	Search page UI — bar, filter panel, results table, location card	2	Dev
3.4	Stock movement forms — IN, OUT, TRANSFER, ADJUSTMENT	2	Dev
3.5	Movement log page — paginated history with filters	1	Dev

Phase 4 — Dashboard & Administration (Week 8–9)

#	Task	Days	Owner
4.1	Dashboard aggregation API — stats, category counts, recent movements	1.5	Dev
4.2	Dashboard UI — stat cards, donut chart, category portfolio cards	2	Dev
4.3	Low-stock alert indicators on dashboard and in parts list	1	Dev
4.4	Location manager page — create/edit zones, racks, shelves, bins	1.5	Dev
4.5	HSN code master page — list, add, edit entries	1	Dev
4.6	User management page — create users, assign roles, toggle active	1.5	Dev

Phase 5 — Testing, Hardening & Deployment (Week 10)

#	Task	Days	Owner
5.1	End-to-end testing: add stock, move stock, search all categories	2	Dev + SecuLogix
5.2	Role permission testing — ADMIN / OPERATOR / VIEWER boundary tests	1	Dev
5.3	Data seed with real SecuLogix parts (provided by client)	1	SecuLogix
5.4	Server setup: Node.js, PM2, Nginx, SSL certificate (Let's Encrypt)	1	Dev
5.5	Configure pg_dump cron job for nightly PostgreSQL backups	0.5	Dev
5.6	User training walkthrough + handover documentation	1	Dev + SecuLogix

11. Deployment Architecture

11.1 Recommended: Self-Hosted (Ubuntu Server)

For maximum data control and zero cloud cost, the recommended deployment is a single Ubuntu 22.04 LTS server:

Component	Details
OS	Ubuntu 22.04 LTS (minimum 4GB RAM, 2 vCPU, 50GB SSD)
Node.js	v18 LTS or v20 LTS — installed via nvm
PostgreSQL	15.x — installed via apt, configured with strong password auth
Process Manager	PM2 — keeps Next.js server alive, auto-restart on crash
Reverse Proxy	Nginx — serves HTTPS, proxies /api and / to PM2
SSL Certificate	Certbot + Let's Encrypt — auto-renewal
Firewall	ufw — allow 22 (SSH), 80 (HTTP redirect), 443 (HTTPS) only
Backups	pg_dump nightly cron → compressed tar → external storage

11.2 Alternative: Vercel + Supabase (Cloud)

If self-hosting is not preferred, a fully managed cloud option:

- Next.js deployed to Vercel (free tier supports up to 100GB bandwidth)
- PostgreSQL on Supabase (free tier: 500MB DB, scales to paid as needed)
- Environment variables set in Vercel project settings
- Supabase provides connection pooling via PgBouncer — add ?pgbouncer=true to DB URL

11.3 Environment Variables (.env.local)

The following environment variables must be configured on the server before first deployment:

Variable	Example / Notes
DATABASE_URL	postgresql://user:pass@localhost:5432/seculogix_db
NEXTAUTH_SECRET	Random 32-char string — use: openssl rand -base64 32
NEXTAUTH_URL	https://inventory.seculogix.com
NODE_ENV	production

12. Security Considerations

Area	Implementation
Password Storage	bcryptjs with cost factor 12 — no plaintext ever stored
Session Tokens	HttpOnly, Secure, SameSite=Strict cookies — JS cannot read them
API Protection	Every /api route checks getServerSession() before processing
Input Validation	Zod schemas on every API endpoint — rejects malformed input before DB
SQL Injection	Prisma ORM uses parameterised queries — zero raw SQL in application code
XSS Prevention	React auto-escapes all output; dangerouslySetInnerHTML never used
CSRF	NextAuth CSRF tokens on all form submissions automatically
Database Access	PostgreSQL only listens on localhost — never exposed to internet
Rate Limiting	Apply Nginx rate limiting on /api/auth to prevent brute force
Dependency Audits	npm audit run on every deployment; Dependabot alerts enabled

Appendix A: Common HSN Codes for Electronics

These HSN codes should be pre-loaded via the seed script. Confirm GST rates with your CA before going live.

HSN Code	Description	Category	GST %
8542	Electronic integrated circuits (ICs, microcontrollers, MCUs)	Electronic	18%
8541	Semiconductor devices — diodes, transistors, thyristors	Electronic	18%
8532	Electrical capacitors (fixed, variable, trimmer)	Electronic	18%
8533	Electrical resistors (fixed, variable, NTC/PTC)	Electronic	18%
8504	Transformers, inductors, coils, chokes, power supplies	Electrical	18%
8536	Switches, relays, fuses, connectors, junction boxes (low voltage)	Electrical	18%
8544	Insulated wires, cables, coaxial cable	Electrical	18%
8471	Computers, computing machines, Raspberry Pi, single board computers	Electronic	18%
8517	Telephone sets, WiFi routers, IoT communication modules	IoT	18%
9026	Sensors for measuring flow, level, temperature, pressure	IoT	18%
8543	Other electrical machines — IoT gateways, signal generators	IoT	18%
8534	Printed circuit boards (bare PCBs)	Electronic	18%
3919	Self-adhesive plates, films, foam tape, kapton tape	General	18%
3926	Plastic fittings, standoffs, cable ties, connectors (plastic)	General	18%
7318	Screws, bolts, nuts, washers (iron/steel fasteners)	General	18%

Appendix B: Development Environment Setup

Follow these steps in order to set up the development environment from scratch:

B.1 Prerequisites

- **Node.js 18+:** Download from nodejs.org or install via nvm (recommended)
- **PostgreSQL 15:** Install from postgresql.org. Default port 5432.
- **Git:** Version control
- **VS Code:** Recommended editor with Prisma and Tailwind IntelliSense extensions

B.2 First-Time Setup Commands

#	Command / Action
1	git clone <repo-url> && cd seculogix-instock
2	npm install
3	cp .env.example .env.local (then fill in your DB credentials)
4	createdb seculogix_db (in PostgreSQL as postgres user)
5	npx prisma migrate dev --name init
6	npx prisma db seed (loads categories, HSN codes, admin user)
7	npm run dev → open http://localhost:3000
8	Login: admin@seculogix.com / (set in seed.ts)

Document Sign-Off

This implementation plan is version 1.0 and covers the complete scope as understood from the initial briefing. Upon receiving client inputs from Section 2, the plan will be updated with confirmed values and a fixed project start date will be agreed.

Signatory	Role	Date
_____	Development Lead	_____
_____	SecuLogix — Technical Lead	_____
_____	SecuLogix — Approver	_____

End of Document — SecuLogix InStock v1.0 Implementation Plan